SOFTWARE REQUIREMENTS SPECIFICATION • PROJECT BLUEPRINT

RepoPulse

AI-powered GitHub Issue Intelligence & Triage Platform

Purpose	Turn a public GitHub repository URL into an evidence-backed issue triage workspace.
Audience	Product / Business Analyst, Data Engineer, Full-Stack Engineer, AI/ML Engineer, QA Engineer, DevOps/MLOps Engineer, Security Reviewer and Maintainers.
Version	Draft 1.0 · 23 October 2026
Document status	Planning baseline — validate scope and acceptance criteria with stakeholders before implementation.

Core product promise

Help maintainers decide what to investigate first, what may be a duplicate or closure candidate, how long comparable issues took to resolve, where investigation might begin, and which contributors have relevant prior experience. RepoPulse provides evidence and recommendations; people remain responsible for consequential decisions.

Design principle: readable, modular, explainable, incremental and safe by default.

Document map

This document is intentionally divided into focused sections. Each engineering role has a separate section with responsibilities, deliverables, dependencies and acceptance checks.

Section	Contents
1	Executive summary and goals
2	Scope, assumptions and boundaries
3	User roles and end-to-end workflow
4	Functional requirements (FR)
5	Non-functional requirements (NFR)
6	System architecture and data flow
7	Data model and API expectations
8	Role: Product / Business Analyst
9	Role: Data Engineer
10	Role: Full-Stack Engineer
11	Role: AI/ML Engineer
12	Role: Code Intelligence / Retrieval Engineer
13	Role: QA / Test Engineer
14	Role: DevOps / MLOps Engineer
15	Role: Security / Privacy Reviewer
16	Cross-role dependencies and delivery plan
17	Acceptance criteria and evaluation
18	Risks, limitations and mitigations
19	Reusable SRS template
20	Glossary and implementation checklist

How to use this document

- Treat FR/NFR identifiers as traceable requirement IDs. Change them only with a documented revision.
- Use the role sections to assign ownership; some tasks require collaboration and should not be treated as isolated silos.
- The first release should deliver reliable ingestion, explainable triage, basic prioritization and an admin dashboard before advanced AI features.
- Replace proposed thresholds and example acceptance criteria with values agreed by the project team after a small pilot.

1. Executive summary

RepoPulse analyzes issues from a public GitHub repository and turns a large backlog into a ranked, searchable, evidence-backed work queue.

Problem statement

Maintainers may face thousands of issues containing duplicates, stale reports, feature requests, incomplete bug reports and urgent defects. Manual triage takes time, while issue lists alone rarely explain urgency, likely resolution duration, relevant code files or contributors with related experience.

Proposed solution

- User submits a public repository URL.
- RepoPulse collects issues and selected related metadata through the GitHub API, with pagination, rate-limit handling and resumable jobs.
- A reusable data pipeline normalizes and stores records; the system does not require per-repository manual cleaning.
- Rules and ML models classify issues, identify possible duplicates, recommend triage actions, prioritize work and estimate resolution outcomes where evidence supports it.
- Retrieval searches similar historical issues, linked fixes, relevant source files and contributors who authored related merged pull requests.
- A dashboard explains recommendations, supports filters and exports, and provides a human-approved workflow for optional GitHub actions.

Primary users

User	Main need
Repository maintainer / admin	Triage backlog, prioritize work, review closure candidates, assign issues and inspect evidence.
Contributor / developer	Understand issue context, locate relevant code/tests and find prior fixes.
Project lead	See backlog trends, resolution patterns and where work is concentrated.
Read-only viewer	Explore public issue analytics without permission to change repository state.

Success statement

Reduce repetitive manual triage and make decisions more evidence-based. Do not promise perfect diagnosis, guaranteed fix times or automatic determination that an issue is unnecessary.

2. Scope, assumptions and boundaries

In scope for the planned product

- Public GitHub repository URL input and validation.
- Collection of issues, labels, timestamps, state, assignees, comments when needed, linked pull requests, and merged PR metadata where accessible.
- Automatic cleaning, deduplication, persistence, incremental refresh and processing status.
- Issue classification, duplicate candidates, stale/resolved/answered review suggestions and explainable prioritization.
- Resolution-time estimation with uncertainty and appropriate treatment of unresolved issues.
- Search for related issues, relevant code files/tests, historical fixes and contributors with related merged work.
- Admin dashboard, sorting/filtering, exports, review queue, audit history and optional authorized assignment/closure actions.

Out of scope for the first release

- Guaranteeing that a predicted root cause is correct or automatically fixing arbitrary code.
- Deleting issues automatically or closing them without an explicit admin action.
- Accessing private repositories without authorization.
- Treating contributor history as proof of availability, responsibility or willingness to accept an issue.
- Supporting every repository language and repository layout with equal quality from day one.
- Training a separate model from scratch every time a user submits a repository URL.

Assumptions

- Initial target is public GitHub repositories; authenticated read access may be added for higher API limits.
- The first MVP is a local or small hosted application using Python, Streamlit, SQLite and a simple vector index.
- Model quality depends on history volume, issue labels, linked PRs, repository language and similarity to training data.
- Predictions are advisory and show their evidence, timestamp, model version and limitations.

Terminology that must remain distinct

Term	Meaning in RepoPulse
Close	Mark an issue as closed. This is not the same as deleting it.
Delete	Remove an issue or data record; destructive and not part of automatic triage.
Duplicate candidate	A likely related/duplicate report that requires confirmation.
Priority	Recommended order of attention, based on impact/urgency and policy.
Risk score	Model estimate for a defined outcome, e.g. still open after 30 days.
Resolution time	Elapsed time between issue creation and closure; open issues are censored observations.

3. User roles and end-to-end workflow

Primary user journey

Stage	System behavior	Visible output
1. Submit URL	Validate URL, repository visibility and access; create analysis job.	Repository name, validation status, job ID.
2. Collect	Fetch paginated issue records and selected metadata; checkpoint progress.	Collection counts, time range, rate-limit and partial-data status.
3. Prepare	Normalize fields, deduplicate, preserve raw source and build searchable records.	Clean dataset and data-quality summary.
4. Analyze	Classify issues, find duplicate candidates, apply triage rules and priority logic.	Categories, recommendations, priority and reasons.
5. Predict / retrieve	Estimate resolution outcomes; retrieve similar issues, files, PRs and contributor history.	Ranges/probabilities, source links and suggested investigation paths.
6. Review	Admin filters results, checks evidence and approves/rejects recommendations.	Review queue and recorded decisions.
7. Optional action	With authorization and confirmation, assign or close an issue through GitHub.	Action result and audit event.
8. Refresh	Fetch only new/updated records and rerun affected analysis.	Updated results and last-refresh timestamp.

User roles

Role	Permissions / behavior
Public viewer	View allowed analytics and source links; no write actions.
Maintainer/admin	Review recommendations and, if authorized, approve assignment or closure actions.
Project operator	Monitor collection jobs, refresh status and failures.
Developer/contributor	Use issue details, similar issues and code investigation suggestions; does not gain extra GitHub permissions through RepoPulse.

4. Functional requirements (FR)

ID	Requirement	Requirement statement	Priority
FR-01	Repository input and validation	The system shall accept a GitHub repository URL, parse owner/repository, validate it and display actionable errors for invalid, deleted, inaccessible or unsupported repositories.	P0
FR-02	Issue collection	The system shall fetch issues with pagination and preserve issue ID/number, title, body, state, labels, timestamps, assignees and source URL. Optional comments/events/PR metadata shall be configurable.	P0
FR-03	Rate limits and resumability	The system shall respect GitHub API limits, retry transient errors with backoff, checkpoint progress and resume interrupted jobs.	P0
FR-04	Incremental refresh	The system shall identify new or changed records and avoid a full re-download when a valid incremental update is possible.	P1
FR-05	Data normalization	The system shall normalize timestamps, labels and missing fields, deduplicate records, exclude pull requests from issue counts, and retain source IDs.	P0
FR-06	Data quality report	The system shall report records collected, skipped records, missing fields, date coverage and incomplete data warnings.	P1
FR-07	Issue classification	The system shall classify issues into configurable categories such as bug, feature, documentation, question and other; it shall expose confidence or an unknown category when uncertain.	P1
FR-08	Duplicate / related issue detection	The system shall rank possible duplicates or related issues and show matching issue links plus similarity evidence.	P1
FR-09	Triage recommendation	The system shall recommend investigate, request information, possible duplicate, possibly resolved, stale, answered or review-for-closure with rationale and supporting evidence.	P1
FR-10	Priority ranking	The system shall rank issues using configurable factors such as severity, impact, urgency, age, dependencies and risk; admins shall be able to filter and sort by these factors.	P1
FR-11	Resolution-time estimate	The system shall estimate resolution-time ranges or time-dependent probabilities when data quality permits and clearly show uncertainty and sample support.	P2
FR-12	Code investigation suggestions	The system shall suggest relevant files, functions, tests, stack-trace locations or historical fixes where evidence is available, and link to the supporting source/version.	P2
FR-13	Contributor recommendations	The system shall rank contributors using relevant prior merged fixes, affected-file familiarity, related issue history and recency; it shall explain the evidence and not infer current availability.	P2
FR-14	Issue detail view	The system shall show the original issue link, classification, recommendation, priority, model evidence, similar issues, file suggestions and contributor suggestions when available.	P1
FR-15	Dashboard search and filters	The system shall support search by issue ID/title/text and filters for state, category, priority, age, risk, recommendation, estimate and label.	P1
FR-16	Sorting	The system shall support sorting by priority, age, risk score, estimated resolution time, duplicate likelihood and contributor match score where available.	P1
FR-17	Admin review queue	The system shall let an authorized admin approve/reject a recommendation, record an optional reason and review past decisions.	P1
FR-18	GitHub write actions	If enabled, the system shall require authorized credentials and explicit confirmation before assigning or closing an issue; every action shall be logged.	P2
FR-19	Export	The system shall export filtered issue/recommendation results to CSV or JSON without leaking secrets or unauthorized data.	P2
FR-20	Model/system monitoring	The system shall expose model version, analysis timestamp, job status, errors and basic evaluation metrics.	P1

5. Non-functional requirements (NFR)

ID	Quality attribute	Requirement
NFR-01	Performance	Show job acknowledgement promptly and run large repository scans asynchronously. Dashboard filters should feel responsive on the supported MVP dataset. Set concrete latency targets after profiling.
NFR-02	Scalability	Use pagination, batching, incremental updates and repository-level partitions/indexes. Avoid loading all issue bodies or embeddings into memory at once.
NFR-03	Reliability	Processing must be idempotent; transient API failures should retry with backoff; jobs should resume from checkpoints and preserve partial results.
NFR-04	Security	Use least-privilege GitHub credentials; keep tokens in environment/secrets storage; redact secrets from logs; protect admin routes and write actions.
NFR-05	Privacy / data minimization	Collect only the fields required for declared features. Document retention and deletion behavior for cached issue content and user credentials.
NFR-06	Explainability	Every consequential recommendation should include a reason, source links, timestamp and model/rule version where relevant.
NFR-07	Auditability	Record admin approvals, rejections, assignments, closures, actor, time and action result without storing secret values.
NFR-08	Maintainability	Use modular packages, typed schemas, configuration, code formatting, linting, automated tests and documented local setup.
NFR-09	Usability	Provide readable status labels, sensible defaults, clear errors, progress indication and a way to inspect the source evidence.
NFR-10	Accessibility	Use sufficient contrast, keyboard-accessible controls, labels for form fields and avoid relying on color alone to communicate risk.
NFR-11	Portability	Support local development on common operating systems and keep configuration external to code. Containerization may be added for deployment.
NFR-12	Cost control	Set collection limits, embedding batches and optional LLM usage controls. Cache results and avoid reprocessing unchanged content.
NFR-13	Model quality	Evaluate on time-separated data; test generalization on repositories not used for training; report class imbalance and calibration where relevant.
NFR-14	Graceful degradation	If code indexing, comments or model predictions are unavailable, still show basic issue data and mark missing features as unavailable rather than failing the whole analysis.
NFR-15	Compatibility	Define supported GitHub API versions, Python version and dependency ranges; pin dependencies for reproducible builds.

Targets to agree before release

Examples to validate during a pilot: maximum repository size for synchronous preview, expected completion time for 500/2,000/10,000 issues, retention period, minimum acceptable quality for closure recommendations, and acceptable API/model costs. Do not claim these targets are met until measured.

6. System architecture and data flow

Logical components

Component	Responsibility	MVP implementation
Frontend	URL input, job progress, dashboard, filters, issue detail and admin review.	Streamlit
API / orchestration	Validate requests, create jobs, expose status/results and coordinate pipeline stages.	FastAPI optional for MVP; direct service calls acceptable initially.
GitHub connector	Paginated collection, rate-limit handling, retries, repository metadata and source links.	GitHub REST API client
Data pipeline	Normalize fields, deduplicate, preserve raw payloads, build clean tables and incremental checkpoints.	Python + Pandas
Storage	Persist issues, labels, jobs, predictions, decisions and action logs.	SQLite for local MVP; PostgreSQL for hosted version.
ML / rules	Classification, duplicate ranking, triage rules, priority and time estimation.	scikit-learn + explicit rule modules
Retrieval / code index	Search issues, source chunks, files, tests, PRs and historical fixes.	BM25 + embeddings + vector index
LLM explanation (optional)	Summarize retrieved evidence and explain recommendations with citations.	Provider abstraction; optional local/cloud model
Observability	Logs, metrics, job status, errors and model versioning.	Python logging; later add metrics/tracing.

Recommended data flow

Step	Component	Behavior
1	User submits URL	Validate URL and create an analysis job.
2	Collector	Fetch issues and metadata in pages; checkpoint each completed batch.
3	Normalizer	Deduplicate and normalize records while preserving original source data.
4	Enrichment	Add linked PR/fix history, code tree metadata and contributor contribution evidence where accessible.
5	Analysis	Run classification, duplicate detection, triage rules, priority ranking and supported estimates.
6	Indexing	Create searchable text/code chunks and metadata for retrieval.
7	Dashboard	Present ranked issues with filters, evidence and progress/quality warnings.
8	Admin action	After authorization and confirmation, call GitHub write endpoint and record audit event.

7. Data model and API expectations

Suggested core tables

Table	Important fields	Purpose
repositories	repo_id, owner, name, url, default_branch, last_synced_at	Repository identity and sync state.
issues	repo_id, issue_id, number, title, body, state, created_at, updated_at, closed_at, url	Canonical issue record; use GitHub numeric ID as stable key.
issue_labels	issue_id, label_name	Many-to-many label mapping.
comments (optional)	comment_id, issue_id, author_id, created_at, body	Evidence and discussion history; collect only if needed.
pull_requests	pr_id, issue_id/linked_issue, author_id, merged_at, url	Links issues to fixes when relationships are available.
changed_files	pr_id, path, change_type, additions, deletions	Evidence for affected modules and contributor expertise.
predictions	issue_id, model_version, prediction_type, score, range, created_at, evidence_refs	Persist model outputs with version and timestamp.
recommendations	issue_id, recommendation_type, rationale, confidence, evidence_refs, status	Admin-reviewable actions and supporting evidence.
contributors	repo_id, user_id, username, contribution_features	Aggregate relevant contribution evidence; do not store secrets.
jobs	job_id, repo_id, stage, status, progress, checkpoint, error_summary	Track long-running analysis and retries.
audit_events	actor, action, issue_id, before_state, after_state, timestamp, result	Trace approvals and GitHub write actions.

Suggested API endpoints if using FastAPI

Method + path	Purpose
POST /repositories/analyze	Validate URL and start analysis job.
GET /jobs/{job_id}	Return status, progress, warnings and errors.
GET /repositories/{repo_id}/issues	Paginated issues with filters and sorting.
GET /issues/{issue_id}	Issue details, recommendations and evidence.
GET /issues/{issue_id}/similar	Similar issues, linked fixes and evidence.
GET /issues/{issue_id}/contributors	Ranked contributor suggestions.
POST /recommendations/{id}/decision	Admin approves/rejects recommendation.
POST /issues/{id}/assign	Optional authorized assignment; require confirmation.
POST /issues/{id}/close	Optional authorized close; require confirmation.
GET /repositories/{repo_id}/metrics	Collection status, trends and evaluation summaries.

8. Role: Product / Business Analyst

Purpose: translate maintainer problems into agreed workflows, measurable requirements and acceptance criteria. This role prevents the team from building features that sound intelligent but are not useful.

Responsibilities

- Identify user groups: repository admin, contributor, project lead and read-only viewer.
- Define issue outcomes and terminology: investigate, request information, duplicate candidate, stale, possibly resolved and review for closure.
- Define what priority means for the selected repository; keep priority separate from predicted resolution risk.
- Agree which metadata may be collected, how much history to fetch and how users understand uncertainty.
- Define permissions and confirmation rules for assignment/closure actions.

Implementation tasks

- Interview 2–3 maintainers or use a documented set of representative workflows.
- Create user stories and acceptance criteria for each FR.
- Define triage policy, default filters, sort behavior and explanation format.
- Agree on pilot repositories, dataset limits, measurable success criteria and release scope.
- Maintain requirement traceability from user need to test case.

Expected deliverables

- Product scope and assumptions document.
- User personas, user stories and workflow diagrams.
- Prioritized backlog with FR IDs and acceptance criteria.
- Decision log for scope, definitions, risks and unresolved questions.

Inputs and collaboration

- Works with all engineering roles; supplies definitions and examples to Data/AI teams.
- Works with QA to make acceptance criteria testable.
- Works with admins to validate recommendations and usability.

Definition of done / acceptance checks

- Every P0/P1 requirement has an owner, testable acceptance criteria and priority.
- Stakeholders agree that closure candidates are recommendations, not automatic deletion.
- Dashboard filters and labels have unambiguous definitions.
- Success metrics are agreed before the pilot begins.

Common risks to avoid

- Mixing priority, severity, risk and resolution time into one unexplained score.
- Promising automatic root-cause diagnosis or guaranteed resolution dates.
- Leaving policy questions, such as stale threshold, undefined.

9. Role: Data Engineer

Purpose: build the reliable ingestion and data foundation. The cleaning pipeline is developed once and reused across compatible repositories; repository-specific differences are handled through schemas and validation, not manual cleaning.

Responsibilities

- Own GitHub API collection, pagination, retry/backoff, rate-limit handling and checkpointing.
- Design raw and normalized data schemas and stable keys.
- Build reusable cleaning/normalization functions and data-quality reports.
- Persist issues, labels, selected comments, PR links, changed files, jobs and sync checkpoints.
- Implement incremental refresh and safe reprocessing.

Implementation tasks

- Build a repository URL parser and repository metadata fetcher.
- Collect issues in pages; distinguish issues from pull requests in API results.
- Normalize timestamps to UTC, handle missing bodies/labels, deduplicate by stable IDs and retain source URLs.
- Store raw payload or a controlled raw snapshot for debugging, subject to retention policy.
- Build data-quality checks: duplicates, missing IDs, malformed timestamps, count reconciliation and sync completeness.
- Provide clean datasets/features to ML and retrieval services.

Expected deliverables

- GitHub client module and ingestion job.
- Schema/migrations and repository/issue tables.
- Cleaning and validation package.
- Retry, checkpoint and incremental-sync tests.
- Data dictionary and sample sanitized dataset.
- Collection report showing records fetched, skipped, updated and warnings.

Inputs and collaboration

- Needs collection scope and field requirements from BA/AI roles.
- Provides stable schema and feature snapshots to AI/ML and code retrieval.
- Provides fixtures and edge cases to QA.
- Coordinates secrets/configuration and job deployment with DevOps.

Definition of done / acceptance checks

- Re-running the same page does not create duplicate issue rows.
- Interrupted jobs resume from a checkpoint without losing completed batches.
- Rate-limit and network failures are surfaced clearly and retried safely.
- Data-quality report identifies missing fields and incomplete coverage.
- Unit tests cover pagination, PR filtering, timestamps and empty/malformed fields.

Common risks to avoid

- Downloading the entire repository history on every run.
- Treating missing descriptions or labels as fatal errors.
- Assuming every repository has the same labels or workflow.
- Discarding source IDs or provenance needed to trace a recommendation.

10. Role: Full-Stack Engineer

Purpose: build the API and user experience that turns analysis outputs into a practical maintainer workflow. For an MVP, Streamlit can cover the UI; FastAPI becomes useful when API separation is needed.

Responsibilities

- Implement repository input, job creation, progress display, dashboard and issue detail pages.
- Expose or consume APIs for repository jobs, issues, metrics, similar issues, contributor recommendations and admin decisions.
- Implement search, filters, sorting, pagination, export and empty/error states.
- Enforce user role/authorization checks for write actions and keep read-only access distinct.
- Make recommendation evidence and uncertainty visible.

Implementation tasks

- Build repository submission form and URL validation feedback.
- Build dashboard cards/charts and issue table with configurable filters.
- Build issue detail panel with triage rationale, linked evidence, relevant files and contributor ranking.
- Build admin review queue with approve/reject and optional reason.
- Integrate job progress, partial results and retry controls.
- Add accessible UI states; do not rely only on red/green colors.

Expected deliverables

- Streamlit UI or frontend app.
- API routes/schemas if using FastAPI.
- Dashboard filters and sorting.
- Issue detail and review queue.
- Error/empty/loading states and export.
- API contract documentation and UI tests.

Inputs and collaboration

- Consumes schemas and endpoints agreed with Data Engineer and AI/ML Engineer.
- Works with BA on user flows and QA on UI/API acceptance.
- Works with DevOps on config, deployment and environment-specific settings.

Definition of done / acceptance checks

- Invalid URL produces a clear message and no invalid job.
- Long-running analysis does not freeze the UI; progress is visible.
- Filters and sort order are consistent and testable.
- Every recommendation shows evidence links and an uncertainty state.
- Write actions require authorization and explicit confirmation.

Common risks to avoid

- Putting data ingestion, ML and UI logic in one giant file.
- Showing fake certainty or hiding incomplete collection warnings.
- Allowing UI controls to bypass server-side authorization.

11. Role: AI/ML Engineer

Purpose: build and evaluate issue intelligence models. Use simple baselines first and only add complexity when it improves measured quality.

Responsibilities

- Develop issue classification, duplicate ranking, triage recommendation support, priority/risk models and resolution-time estimation.
- Define labels/targets with the BA and prevent leakage from future information.
- Create evaluation datasets, baselines, metrics and model/version tracking.
- Expose confidence/uncertainty and fallback states; avoid unsupported claims.
- Monitor generalization across repositories and issue types.

Implementation tasks

- Start with TF-IDF + Logistic Regression for issue categories and baseline text similarity.
- Create time-aware train/validation/test splits; test on repositories not used during training where possible.
- Define the 30-day unresolved target carefully; exclude issues without sufficient follow-up or use survival analysis.
- Evaluate macro-F1, per-class precision/recall, PR-AUC where appropriate and probability calibration.
- Develop a transparent priority baseline; avoid training a model before reliable labels exist.
- Produce explanations tied to features/evidence; never use future comments or closing dates as prediction-time inputs.
- Compare model recommendations with maintainer judgments on a reviewed sample.

Expected deliverables

- Training and inference pipeline.
- Versioned model artifacts and feature schema.
- Evaluation report with baselines, metrics, errors and limitations.
- Prediction API/service and model card.
- Threshold/calibration notes and monitoring recommendations.

Inputs and collaboration

- Needs clean data and timestamps from Data Engineer.
- Provides prediction schema and confidence/evidence contract to Full-Stack Engineer.
- Works with QA on leakage tests and reproducibility.
- Works with DevOps/MLOps on artifact storage, versioning and rollback.

Definition of done / acceptance checks

- Test data is time-separated and not used in training or tuning.
- Model report includes class imbalance, confusion matrix and relevant metrics.
- Predictions include model version and generated timestamp.
- Uncertain or unsupported cases return an explicit fallback instead of fabricated confidence.
- Results are compared against a simple baseline and documented.

Common risks to avoid

- Using future closure status, future comments or later merged PRs as features for a prediction made at issue creation.
- Reporting accuracy alone on an imbalanced dataset.
- Treating currently open issues as permanent failures.
- Claiming an LLM explanation is a verified root cause.

12. Role: Code Intelligence / Retrieval Engineer

Purpose: help maintainers find related reports, likely code locations, historical fixes and relevant contributors. This can be owned by the AI/ML Engineer in a small team, but it deserves a separate workstream because it needs different indexing and evaluation.

Responsibilities

- Index issue text, selected discussions, source-code chunks, tests, pull request descriptions and changed-file metadata.
- Build hybrid retrieval using lexical search and dense embeddings; optionally rerank results.
- Preserve provenance: repository, commit SHA, file path, line range, issue/PR URL and timestamps.
- Connect similar historical issues to merged PRs and contributors who implemented fixes.
- Return evidence-backed suggestions and indicate when no reliable match exists.

Implementation tasks

- Chunk code by file/function where possible; avoid arbitrary oversized chunks.
- Use metadata filters for repository, branch/commit, language, file path and record type.
- Retrieve top-k similar issues and evaluate with manually reviewed relevance labels.
- Search stack-trace paths, error messages, function names and test failures when present.
- Use changed-file lists from merged PRs to identify relevant contributors; distinguish PR author from reviewers/commenters.
- Provide source links and exact code version for each suggestion.

Expected deliverables

- Indexing and retrieval pipeline.
- Hybrid search endpoint/service.
- Code/issue chunk schema and metadata rules.
- Evidence formatter with file/line/PR links.
- Retrieval evaluation report and curated examples.

Inputs and collaboration

- Needs issue/PR/file data from Data Engineer.
- Works with AI/ML Engineer on embeddings, ranking and LLM summarization.
- Provides result schemas to Full-Stack Engineer.
- Coordinates repository access and indexing resource limits with DevOps.

Definition of done / acceptance checks

- Every displayed result has a valid source URL or clearly states that provenance is unavailable.
- Top-k relevance is tested against reviewed examples.
- File suggestions are linked to a commit SHA or branch and are not presented as confirmed root cause.
- Missing code access does not break issue-only analysis.
- Contributor ranking uses relevant merged work, not comment count alone.

Common risks to avoid

- Indexing the whole codebase as one text blob.
- Assuming semantically similar issues share the same root cause.
- Linking line numbers from an old commit as though they were current.
- Treating contributors' historical work as proof they are available.

13. Role: QA / Test Engineer

Purpose: verify the application end-to-end, including difficult GitHub data, model uncertainty, permissions and safe handling of admin actions.

Responsibilities

- Create test strategy, test cases, fixtures and acceptance tests mapped to FR/NFR IDs.
- Test data ingestion, API/UI behavior, filters, sorting, retries, job resume and error states.
- Validate model evaluation methodology and check for leakage.
- Test that evidence links are correct and recommendations are not silently converted into actions.
- Run regression tests before releases.

Implementation tasks

- Unit-test URL parsing, normalizers, deduplication and rules.
- Integration-test GitHub API pagination with mocked responses, rate limits, empty repositories and deleted/inaccessible repos.
- Test large result sets, malformed payloads, missing labels and unusual timestamp values.
- Test admin vs viewer permissions; verify write action confirmation and audit logging.
- Create a manually reviewed golden set for duplicate, triage and file suggestion evaluations.
- Test model reproducibility and report quality metrics.

Expected deliverables

- Test plan and traceability matrix.
- Unit/integration/end-to-end test suites.
- Golden evaluation set and labeling guide.
- Bug reports with reproduction steps.
- Release test summary and sign-off.

Inputs and collaboration

- Needs acceptance criteria from BA and testable contracts from engineers.
- Works with Data Engineer on data edge cases.
- Works with AI/ML and retrieval roles on offline evaluation.
- Works with DevOps on CI and release gates.

Definition of done / acceptance checks

- Every P0 requirement has at least one test.
- Critical paths cover happy path, invalid input, partial failure and retry.
- No unauthorized user can perform assignment/closure.
- Known limitations and incomplete-data states are tested.
- Regression suite runs in CI and results are reproducible.

Common risks to avoid

- Testing only happy paths with a single small repository.
- Using synthetic model metrics as though they were real evaluation results.
- Accepting a high score without checking supporting evidence.

14. Role: DevOps / MLOps Engineer

Purpose: make RepoPulse reproducible, observable, secure to configure and straightforward to run locally or deploy.

Responsibilities

- Provide environment configuration, dependency pinning, repeatable builds and deployment workflow.
- Manage secrets securely and separate read-only GitHub analysis from optional write permissions.
- Monitor job duration, failure rate, API usage, model version and system resource consumption.
- Store model/index artifacts with version identifiers and support rollback.
- Set resource limits for large repositories and optional LLM/embedding services.

Implementation tasks

- Create reproducible setup instructions, requirements/lock file and optional Docker image.
- Configure CI for lint, type checks, unit/integration tests and dependency/security scans.
- Implement structured logs and health/readiness endpoints if using an API server.
- Define backup/retention policy for database, raw snapshots and vector index.
- Configure queue/worker execution for large repositories and job concurrency limits.
- Add model/index versioning and safe re-index/rollback strategy.

Expected deliverables

- Environment templates with no real secrets.
- CI workflow and deployment configuration.
- Logging/health checks and job metrics.
- Backup/restore and retention notes.
- Runbook for common API, rate-limit and indexing failures.

Inputs and collaboration

- Needs resource and deployment target decisions from BA/team.
- Works with Full-Stack on runtime/API config.
- Works with Data Engineer on workers, storage and backups.
- Works with AI/ML/retrieval on artifact and index versioning.

Definition of done / acceptance checks

- A new developer can follow README and reproduce the application.
- CI blocks merge when required tests fail.
- Secrets do not appear in repository or logs.
- Failed jobs expose useful error summaries and can be retried.
- Deployment and rollback steps are documented.

Common risks to avoid

- Committing tokens or local paths.
- Unbounded parallel collection or embedding jobs.

- Rebuilding every index/model on every page load.
- Failing silently when a model provider or vector store is unavailable.

15. Role: Security / Privacy Reviewer

Purpose: reduce risk around credentials, repository actions, stored issue content and AI-generated recommendations. In a small team this may be a responsibility shared by the tech lead and DevOps Engineer.

Responsibilities

- Review authentication, authorization, secret handling, data retention and external API calls.
- Ensure public read access does not imply permission to write to GitHub.
- Review prompt-injection risk in issue bodies, comments, source files and other untrusted repository content.
- Define safeguards for assignment/closure operations and audit logs.
- Ensure retrieved repository text is treated as data, not as trusted instructions to the system.

Implementation tasks

- Use least-privilege OAuth/GitHub App permissions and separate read and write capabilities.
- Validate API input and prevent server-side request forgery through arbitrary URL handling; allow only expected GitHub URL patterns.
- Redact tokens from logs and errors; store secrets in environment/secret manager.
- Require admin confirmation for writes and verify repository/user authorization server-side.
- Set retention and deletion policies for cached content and derived indexes.
- Test prompt injection and malicious issue text; constrain LLM tools and validate outputs against schemas.

Expected deliverables

- Threat model and security checklist.
- Permission matrix and secret-handling policy.
- Write-action approval design and audit requirements.
- Prompt-injection and malicious-input test cases.
- Retention and incident-response notes.

Inputs and collaboration

- Works with all roles; especially Full-Stack, DevOps and retrieval/LLM implementers.
- Reviews API scopes and deployment configuration before write actions are enabled.

Definition of done / acceptance checks

- No secrets in source control or logs.
- Read-only mode works without write permissions.
- Write actions are denied without explicit authorization and confirmation.
- Untrusted issue text cannot trigger arbitrary tools or override system policy.
- Audit events contain action metadata but no credential values.

Common risks to avoid

- Treating public repository text as trusted instructions.
- Allowing the LLM to execute GitHub actions without a constrained approval path.

- Collecting more user or repository data than the feature requires.

16. Cross-role dependencies and delivery plan

Role dependency map

Workstream	Primary owner	Key dependencies / consumers
Requirements and policy	Product / Business Analyst	All roles; defines labels, thresholds, admin actions and acceptance criteria.
GitHub ingestion + normalized data	Data Engineer	Feeds ML, retrieval, UI and QA.
Dashboard + API contracts	Full-Stack Engineer	Depends on stable schemas from Data/AI; tests with QA.
Classification + time estimates	AI/ML Engineer	Depends on labeled historical data; exposes model outputs to UI.
Code + contributor retrieval	Code Intelligence Engineer	Depends on issue/PR/file history; provides evidence to AI and UI.
Testing + evaluation gates	QA Engineer	Needs acceptance criteria and reproducible data/model artifacts.
Deployment + observability	DevOps/MLOps Engineer	Supports ingestion jobs, storage, model/index versioning and CI.
Permissions + threat review	Security Reviewer / Tech Lead	Reviews credentials, prompt injection, write actions and retention.

Suggested staged roadmap

Phase	Scope	Exit condition
1. Foundation	URL validation, collector, normalized schema, job status and basic dashboard.	Can analyze one supported public repository and resume a failed collection.
2. Triage MVP	Issue categories, duplicate candidates, basic rules, filters and evidence links.	Maintainer can review and correct recommendations; quality is measured on a small labeled sample.
3. Prioritization	Priority score/rules, baseline ML and time-aware evaluation.	Model report includes baselines, leakage checks and limitations.
4. Investigation	Similar issues, linked fixes, file suggestions and contributor ranking.	Top-k suggestions are evaluated and source links are shown.
5. Controlled actions	Optional assign/close workflow, authorization, confirmation and audit trail.	Security tests pass; no silent or unauthorized write actions.
6. Production hardening	CI/CD, monitoring, backups, incremental refresh, performance testing.	Operational runbook, rollback and acceptance sign-off complete.

17. Acceptance criteria and evaluation

Functional acceptance examples

- Given a valid public GitHub URL, the system creates a job and reports progress without requiring manual CSV preparation.
- Given an interrupted collection, the system resumes from saved checkpoints and does not duplicate records.
- Given an issue flagged as a duplicate, the UI shows at least one candidate source issue and explains why it was matched.
- Given a closure recommendation, the UI labels it as a review candidate and does not close or delete the issue automatically.
- Given a prediction, the result includes the model version, timestamp and the target being predicted.
- Given a file suggestion, the UI shows repository path and source version/commit when available; otherwise it labels the location as uncertain.
- Given a contributor recommendation, the UI shows evidence from relevant merged work and does not claim the contributor is available.
- Given a read-only user, assignment and closure endpoints reject the request.

Suggested evaluation metrics

Feature	Metric / method	Important caveat
Issue classification	Macro-F1, per-class precision/recall, confusion matrix.	Labels may be noisy or repository-specific.
Duplicate detection	Precision@k / recall on reviewed duplicate pairs.	Similar wording does not always mean duplicate.
Closure-review suggestions	Precision of recommendations accepted by maintainers; false-positive review.	Treat this as decision support, not automated closure.
Priority ranking	Top-k recall for severe issues; maintainer agreement; review-time study.	Priority definitions differ by project.
Resolution-time estimate	MAE/median absolute error on resolved issues; interval coverage; survival metrics.	Open issues are right-censored, not known failures.
Code-location suggestions	Top-k file hit rate on issues with known fix PRs; source validity.	A file touched in a fix is not always the root cause.
Contributor ranking	Top-k match against historical fix contributors; admin feedback.	Past contribution does not imply availability.
Ingestion reliability	Success rate, completeness checks, retry recovery, job duration.	Account for rate limits and unavailable records.

Evaluation rules

- Use time-based train/test splits and, where feasible, hold out entire repositories to test generalization.
- Do not use information that would only exist after the prediction timestamp.
- Manually label a modest golden dataset for duplicates, triage and file suggestions before making quality claims.
- Publish actual measured results and known failure cases; never fill the report with invented scores.

18. Risks, limitations and mitigations

Risk	Impact	Mitigation
GitHub rate limits / partial data	Incomplete analysis or slow jobs.	Backoff, checkpointing, incremental sync, progress warnings and optional authenticated read access.
Repository-specific labels/workflows	Poor category/priority quality.	Configurable label mapping, unknown class, repository metadata and local validation.
False closure recommendation	Useful issue may be closed incorrectly.	Human review, evidence links, reversible close action where possible, no automatic deletion.
Prediction leakage	Inflated offline metrics; poor real-world performance.	Time-aware feature snapshots, leakage tests and held-out repository evaluation.
Unresolved issues lack outcome	Biased resolution estimates.	Censoring-aware evaluation and survival analysis for advanced versions.
Wrong code-file suggestion	Developer wastes time.	Ranked suggestions, exact provenance, commit SHA, confidence and no-match state.
Contributor unfairness / overload	Repeatedly assigning to a few people or implying availability.	Use relevance evidence, avoid volume-only ranking, admin override and workload/availability signals only when legitimate.
Prompt injection in repository content	LLM may follow malicious instructions in issues/comments/code.	Treat content as untrusted data, isolate tools, constrain outputs and require approval for actions.
Large repository costs	Slow indexing and high embedding/model cost.	Configurable limits, incremental indexes, batch processing, caching and optional advanced features.
Stale cached data	Dashboard differs from GitHub current state.	Show last sync, refresh action, updated_at and partial/stale warning.

19. Reusable SRS template

Copy this section for a formal project SRS. Replace bracketed prompts and retain requirement IDs for traceability.

19.1 Document control

- Project name: [name]
- Version / date / author: [version, date, owner]
- Reviewers / approval: [stakeholders]
- Change history: [version, change, author, date]

19.2 Purpose and scope

- Purpose: [problem the software solves]
- In scope: [capabilities included in this release]
- Out of scope: [explicit exclusions]
- Success criteria: [measurable outcomes]

19.3 Stakeholders and users

- Stakeholder / role: [who]
- Goals and permissions: [what they need and may do]
- Assumptions and constraints: [technical, legal, schedule, budget]

19.4 System context and interfaces

- External systems/APIs: [name, purpose, auth, limits]
- User interface: [screens and workflows]
- Data interfaces: [formats, schemas, import/export]
- Deployment environment: [local/cloud/OS/runtime]

19.5 Functional requirements

- ID: FR-[NN]
- Title: [short name]
- Requirement: The system shall [one testable behavior].
- Priority: [P0/P1/P2]
- Preconditions: [what must be true]
- Trigger / main flow: [steps]
- Alternate/error flow: [failure behavior]
- Acceptance criteria: Given [context], when [action], then [observable result].
- Dependencies: [other FRs or teams]

19.6 Non-functional requirements

- ID: NFR-[NN] / quality attribute: [performance/security/reliability/etc.]
- Requirement: [measurable quality constraint]
- Measurement method: [how verified]

- Target: [agreed value; do not invent before profiling]
- Environment and conditions: [dataset/load/platform]

19.7 Data and model requirements

- Entities / fields / keys: [schema]
- Data source / rights / retention: [source and policy]
- Quality checks: [missing values, duplicates, completeness]
- Model target and prediction timestamp: [precise definition]
- Training/evaluation split: [time/repository strategy]
- Metrics and release threshold: [agreed values]
- Explainability/provenance: [evidence and source links]

19.8 Security, privacy and operations

- Authentication / authorization: [roles and scopes]
- Secret storage / logging: [controls]
- Audit events: [actions to record]
- Backup / restore / retention: [policy]
- Monitoring / alerting / runbook: [operations]

19.9 Verification and traceability

- Requirement ID -> design component -> implementation task -> test case -> result
- Test levels: [unit, integration, end-to-end, security, performance, model evaluation]
- Known limitations and residual risks: [list]
- Release approval: [owner/date]

20. Glossary and implementation checklist

Term	Plain-English meaning
FR / NFR	Functional requirement / non-functional requirement.
Triage	Reviewing and routing issues into appropriate next actions.
Embedding	A numeric representation of text/code used for semantic similarity search.
Hybrid search	Combining keyword search with semantic/vector search.
RAG	Retrieval-augmented generation: retrieve relevant evidence before asking an LLM to summarize or explain.
Data leakage	Using information unavailable at prediction time, making evaluation unrealistically optimistic.
Censoring	A time-to-event observation whose final resolution time is not yet known, e.g. an issue still open.
Provenance	Where a claim came from: URL, commit, file path, issue/PR and timestamp.
Idempotent job	A repeated run does not create duplicate or inconsistent results.
Human-in-the-loop	A person reviews and approves consequential recommendations/actions.

Before coding checklist

- Choose 2–3 pilot repositories with enough public issue and merged PR history.
- Agree what counts as a duplicate, stale issue, answered question and high priority.
- Choose an initial collection cap and define incremental refresh behavior.
- Build ingestion and data-quality reporting before training models.
- Create a manually reviewed evaluation sample and record its labeling rules.
- Ship a simple dashboard before adding LLM summaries or GitHub write actions.
- Keep source links, model versions, uncertainty and admin decisions visible.
- Add assignment/closure actions only after permissions, confirmation and audit tests pass.

Recommended MVP definition

A user submits a public GitHub repository URL and receives a dashboard with collected issue data, issue categories, possible duplicates, evidence-backed triage suggestions, configurable priority filters, source links and collection-quality warnings. Resolution-time prediction, code-location suggestions, contributor ranking and write actions are subsequent milestones gated by evaluation and safety checks.

End of document · RepoPulse SRS & Role-wise Blueprint · Draft v1.0